

Pontificia Universidad
JAVERIANA
Colombia

SRS

Presentan

Andres Henao Niño
Joan Emmanuel Orduz Chia
Joshua Prieto Zambrano
David Estevan Rodriguez Jurado

Director
Jaime Andrés Pavlich Mariscal

Pontificia Universidad Javeriana
Facultad de Ingeniería
Trabajo de Grado

Bogotá D.C.
octubre de 2026
Versión 1.1

Historial de Cambios

Versión	Fecha	Descripción del cambio	Autor(es)
1.0	mayo de 2026	Versión inicial en procesador de texto (Word).	Andres Henao Niño, Joan Emmanuel Orduz Chia, Joshua Prieto Zambrano, David Estevan Rodriguez Jurado
1.1	octubre de 2026	Paso a LaTeX con el estilo común, sin cambios de contenido (issue #168).	Andres Henao Niño

Tabla de Contenidos

1. Introducción 1

2. Visión del Producto 1

2.1 Descripción del producto 1

2.2 Alcance 2

2.3 Objetivo del producto 2

3. Restricciones del Producto 2

3.1 Restricciones técnicas 2

3.2 Restricciones regulatorias 2

3.3 Restricciones de alcance del proyecto 3

4. Modelo de Dominio 3

4.1 Entidades del dominio 3

4.2 Relaciones entre entidades 3

5. Características de los Usuarios 3

5.1 Desarrollador de software junior 6

5.2 Desarrollador de software senior 6

5.3 Evaluador / Tester de software 6

6. Funciones del Producto 6

6.1 Requisitos Funcionales 6

6.2 Historias de Usuario 7

6.3 SDK — Interfaz unificada de integración 8

6.3.1 HU-01: Crear una intención de pago con una pasarela específica 8

6.3.2 HU-02: Consultar el estado de una transacción 8

6.3.3 HU-03: Validar y procesar un webhook entrante 8

6.3.4 HU-04: Cambiar de pasarela sin modificar la lógica de negocio 9

6.3.5 HU-05: Manejo automático de reintentos ante fallos transitorios 9

6.4 API web de simulación 9

6.4.1 HU-06: Simular una transacción aprobada 9

6.4.2 HU-07: Simular escenarios de error controlados 10

6.4.3 HU-08: Recibir una notificación webhook simulada 10

6.5 Documentación de datos de prueba 11

6.5.1 HU-09: Acceder a datos de prueba por pasarela 11

7. Requisitos No Funcionales 11

7.1 Funcionalidad (Functionality) 11

7.2 Usabilidad (Usability) 11

7.3 Confiabilidad (Reliability) 12

7.4 Rendimiento (Performance) 12

7.5 Soporte y mantenibilidad (Supportability) 12

7.6 Restricciones adicionales (+) 13

8. Referencias 13

Lista de Tablas

Tabla 1. Entidades del modelo de dominio de Kit Pagos Colombia. 4
Tabla 2. Perfil del desarrollador junior. 4
Tabla 3. Perfil del desarrollador senior. 5
Tabla 4. Perfil del evaluador/tester. 5
Tabla 5. Requisitos funcionales del framework Kit Pagos Colombia. 7
Tabla 6. HU-01 8
Tabla 7. HU-02 8
Tabla 8. HU-03 9
Tabla 9. HU-04 9
Tabla 10. HU-05 9
Tabla 11. HU-06 10
Tabla 12. HU-07 10
Tabla 13. HU-08 10
Tabla 14. HU-09 11

Lista de Figuras

1. Introducción

Este documento corresponde a la Especificación de Requisitos de Software (SRS) del proyecto Kit Pagos Colombia, un framework de código abierto orientado a simplificar el proceso de integración de múltiples pasarelas de pago en aplicaciones de software desarrolladas en el contexto colombiano.

El documento está organizado en las siguientes secciones principales:

- **Visión del producto:** describe el propósito central del framework, su alcance y el problema que busca resolver.
- **Restricciones del producto:** enumera las limitaciones técnicas, regulatorias y de alcance que condicionan las decisiones de diseño e implementación.
- **Modelo de dominio:** presenta los conceptos centrales del dominio de pagos digitales y sus relaciones, con independencia de cualquier implementación específica.
- **Características de los usuarios:** identifica los perfiles de usuario que interactuarán con el framework y describe sus características relevantes.
- **Funciones del producto:** especifica los requisitos funcionales del sistema mediante historias de usuario y una descripción detallada de las funcionalidades esperadas por componente.
- **Requisitos no funcionales:** describe los atributos de calidad que el framework debe satisfacer, siguiendo la clasificación FURPS+.

El documento se apoya en el estándar IEEE 830-1998 (IEEE Recommended Practice for Software Requirements Specifications) como referencia para la estructura y el nivel de detalle de la especificación. Su audiencia principal son los miembros del equipo de desarrollo, el director del proyecto y los evaluadores académicos del trabajo de grado.

2. Visión del Producto

2.1 Descripción del producto

Kit Pagos Colombia es un framework de desarrollo de software de código abierto,cuyo propósito es unificar la integración de las principales pasarelas de pago utilizadas en el mercado colombiano bajo una interfaz común, extensible y reutilizable.

El framework está compuesto por tres componentes complementarios:

- **SDK con interfaz unificada:** biblioteca que abstrae las diferencias técnicas entre Wompi, PayU, Mercado Pago y Kushki, exponiendo una API normalizada para operaciones de creación de pagos, consulta de estado y gestión de notificaciones asíncronas (webhooks).
- **API web de simulación desplegada:** servicio REST accesible públicamente que replica el comportamiento de las pasarelas de pago para los escenarios de prueba más frecuentes, sin requerir credenciales reales ni configuración local adicional.
- **Documentación centralizada de datos de prueba:** repositorio estructurado que consolida los datos de prueba provistos por cada pasarela (tarjetas de prueba, credenciales de sandbox, referencias de pago), organizados por proveedor y tipo de dato para facilitar su acceso durante el desarrollo.

2.2 Alcance

El framework cubre las siguientes operaciones sobre las pasarelas integradas: creación de intenciones de pago, consulta del estado de una transacción, recepción y validación de notificaciones webhook, y normalización de respuestas de error. Las pasarelas soportadas en la versión inicial son Wompi, PayU Latam, Mercado Pago Colombia y Kushki.

El framework NO cubre: procesamiento directo de datos de tarjeta de crédito, modificación de los mecanismos de seguridad internos de cada pasarela, gestión de cuentas de comercio, conciliación financiera, ni operaciones administrativas sobre las cuentas de los proveedores.

2.3 Objetivo del producto

Reducir la complejidad técnica y el tiempo requerido para integrar múltiples pasarelas de pago en aplicaciones de software colombianas, ofreciendo una capa de abstracción reutilizable, un entorno de simulación accesible y una fuente centralizada de datos de prueba válidos para el contexto local.

3. Restricciones del Producto

3.1 Restricciones técnicas

- El SDK se implementa exclusivamente en TypeScript/Node.js. No se desarrollarán versiones para otros lenguajes o entornos de ejecución en el alcance de este proyecto. VER CUAL PUEDE SER EL ALCANCE DE ESTO
- La API de simulación se distribuye como servicio web desplegado. No se ofrece una versión ejecutable localmente mediante contenedores Docker en esta versión.
- El framework no almacena ni procesa datos de tarjeta de crédito directamente. Toda operación sensible es delegada a los mecanismos de seguridad certificados de cada pasarela.
- La comunicación entre el cliente y el framework debe realizarse a través de HTTP. No se soportan conexiones sin cifrado.
- El framework depende de la disponibilidad de los ambientes sandbox de cada pasarela para su validación. Cambios en las APIs de los proveedores pueden requerir actualizaciones en los adaptadores.

3.2 Restricciones regulatorias

- El framework debe operar dentro de los límites establecidos por el estándar PCI DSS para el manejo de datos sensibles, delegando el procesamiento de información de pago a los mecanismos certificados de cada proveedor.
- El uso del framework en entornos de producción está sujeto al cumplimiento de los términos y condiciones de uso de las APIs de cada pasarela (Wompi, PayU, Mercado Pago, Kushki).
- El framework no modifica ni intercepta los flujos de autenticación internos de las pasarelas, en cumplimiento de las políticas de seguridad de cada proveedor.

3.3 Restricciones de alcance del proyecto

- El proyecto se desarrolla en un período de cinco meses (julio a noviembre de 2026), lo que limita la cantidad de pasarelas integradas y la profundidad de cobertura funcional por proveedor.
- La evaluación del framework se realizará sobre proyectos prototípicos definidos, no sobre sistemas en producción con volúmenes reales de transacciones.
- El mantenimiento y evolución del framework más allá del período académico no están garantizados por el equipo de desarrollo.

4. Modelo de Dominio

El dominio del problema se organiza alrededor del concepto de transacción de pago como entidad central, la cual es procesada por una pasarela de pago a través de métodos de pago específicos y genera notificaciones asíncronas hacia la aplicación del comercio. A continuación se describen los conceptos principales del dominio y sus relaciones.

4.1 Entidades del dominio

4.2 Relaciones entre entidades

- Un Comercio se autentica ante una o varias Pasarelas de pago mediante credenciales específicas por proveedor.
- Una Transacción es creada por un Comercio y procesada por exactamente una Pasarela de pago a través de un Método de pago.
- Una Transacción tiene exactamente un Estado de transacción en cada momento de su ciclo de vida.
- Una Pasarela de pago envía cero o más Webhooks hacia el Comercio a lo largo del ciclo de vida de una Transacción.
- El framework expone un Puerto que es implementado por uno o más Adaptadores, uno por cada Pasarela soportada.
- Cada operación fallida genera un Error normalizado que abstrae el error original de la Pasarela.

5. Características de los Usuarios

Se identifican tres perfiles de usuario que interactúan con el framework Kit Pagos Colombia, diferenciados por su nivel de experiencia técnica, su frecuencia de uso y su relación con los componentes del sistema.

Entidad	Descripción
Transacción	Unidad atómica de intercambio económico entre un comprador y un comercio, procesada a través de una pasarela de pago. Tiene un identificador único, un monto, una moneda, un estado y una marca temporal de creación.
Estado de transacción	Representa la condición actual de una transacción en el ciclo de vida del pago. Los estados normalizados del framework son: PENDIENTE, APROBADO, RECHAZADO, EXPIRADO y ERROR.
Pasarela de pago	Proveedor externo que procesa las transacciones de pago. Cada pasarela define sus propias convenciones de API, autenticación, formatos de respuesta y mecanismos de notificación. Las pasarelas soportadas son Wompi, PayU, Mercado Pago y Kushki.
Adaptador	Componente del framework que traduce el contrato normalizado del puerto hacia las convenciones específicas de una pasarela determinada. Existe un adaptador por cada pasarela soportada.
Puerto	Interfaz abstracta del framework que define el contrato que cada adaptador debe cumplir. El puerto especifica las operaciones disponibles (crear pago, consultar estado, validar webhook) sin conocimiento de ningún proveedor específico.
Método de pago	Mecanismo a través del cual el comprador efectúa el pago. Puede ser tarjeta de crédito/débito, transferencia bancaria (PSE), billetera digital u otros métodos soportados por cada pasarela.
Webhook	Notificación asíncrona que una pasarela envía hacia una URL registrada por el comercio para informar sobre cambios en el estado de una transacción. Cada pasarela define su propio formato y mecanismo de firma.
Error normalizado	Representación unificada de una condición de fallo durante el procesamiento de una operación, independiente del formato de error específico de cada pasarela. Incluye un código de error, un mensaje descriptivo y el proveedor que originó el fallo.
Comercio	Entidad que ofrece bienes o servicios e integra el framework para aceptar pagos digitales. Posee credenciales de autenticación para cada pasarela que desea utilizar.
Desarrollador	Persona técnica que consume el framework para integrar capacidades de pago en la aplicación del comercio. Es el usuario principal del SDK y de la API de simulación.

Tabla 1. Entidades del modelo de dominio de Kit Pagos Colombia.

Característica	Descripción
Descripción	Desarrollador con menos de dos años de experiencia profesional que se enfrenta por primera vez a la integración de una pasarela de pago. Puede ser estudiante universitario, freelancer o parte de un equipo de una startup en etapa temprana.
Experiencia técnica	Conocimiento básico de REST APIs y JavaScript/TypeScript. Poca o nula experiencia previa con pasarelas de pago colombianas.
Frecuencia de uso	Esporádica. Utiliza el framework durante el desarrollo inicial de un proyecto y ocasionalmente para agregar nuevas pasarelas.
Necesidades principales	Documentación clara, ejemplos de código funcionales, datos de prueba accesibles y un entorno de simulación que no requiera configuración compleja.
Interacción con el framework	Principalmente con el SDK (operaciones básicas de pago) y la documentación de datos de prueba. Hace uso intensivo de la API de simulación para pruebas locales.

Tabla 2. Perfil del desarrollador junior.

Característica	Descripción
Descripción	Desarrollador con experiencia en integraciones de sistemas y arquitecturas de software. Ha trabajado previamente con APIs de pago o servicios externos. Puede ser parte de un equipo de producto o consultora tecnológica.
Experiencia técnica	Dominio avanzado de TypeScript, Node.js, patrones de diseño y arquitecturas de integración. Familiaridad con los conceptos de autenticación, webhooks y manejo de errores en APIs externas.
Frecuencia de uso	Regular. Utiliza el framework como parte de proyectos activos y puede contribuir a su extensión con nuevos adaptadores.
Necesidades principales	Flexibilidad arquitectónica, capacidad de extensión para agregar nuevas pasarelas, control sobre el comportamiento de reintentos y manejo avanzado de errores.
Interacción con el framework	Con todos los componentes del framework, incluyendo la implementación de adaptadores personalizados y la configuración avanzada del SDK.

Tabla 3. Perfil del desarrollador senior.

Característica	Descripción
Descripción	Persona responsable de verificar el correcto funcionamiento de la integración de pagos en una aplicación. Puede ser un QA engineer, un desarrollador en fase de pruebas o un integrante del equipo de proyecto encargado de validar los flujos de pago antes de salir a producción.
Experiencia técnica	Conocimiento funcional de los flujos de pago. Experiencia con herramientas de prueba de APIs (Postman, curl). No necesariamente escribe código de integración.
Frecuencia de uso	Intensiva durante los ciclos de prueba del proyecto. Uso principalmente de la API de simulación y la documentación de datos de prueba.
Necesidades principales	Acceso a escenarios de prueba predefinidos (éxito, rechazo, error de red, timeout), datos de prueba válidos por pasarela y una API de simulación estable y predecible.
Interacción con el framework	Principalmente con la API de simulación a través de herramientas REST y con la documentación centralizada de datos de prueba.

Tabla 4. Perfil del evaluador/tester.

5.1 Desarrollador de software junior

5.2 Desarrollador de software senior

5.3 Evaluador / Tester de software

6. Funciones del Producto

Esta sección especifica los requisitos funcionales del framework Kit Pagos Colombia. La sección 5.1 presenta los requisitos funcionales en formato tabular con identificador, descripción, componente y prioridad. La sección 5.2 en adelante complementa esta especificación mediante historias de usuario que expresan cada requisito desde la perspectiva del usuario, acompañadas de criterios de aceptación verificables.

6.1 Requisitos Funcionales

La Tabla 5 presenta el listado consolidado de requisitos funcionales del framework, organizados por componente. Las prioridades se clasifican en Alta (esencial para el MVP), Media (importante pero no bloqueante) y Baja (deseable en versiones futuras).

ID	Descripción del requisito	Componente	Prioridad
RF-01	El sistema debe permitir crear una intención de pago especificando pasarela, monto, moneda, referencia de orden y datos del pagador mediante una interfaz unificada.	SDK	Alta
RF-02	El sistema debe retornar una respuesta normalizada ante la creación de un pago, incluyendo el identificador de transacción, el estado inicial y la URL o token de pago según la pasarela.	SDK	Alta
RF-03	El sistema debe permitir consultar el estado de una transacción por identificador y retornarlo mapeado a los estados normalizados: PENDIENTE, APROBADO, RECHAZADO, EXPIRADO o ERROR.	SDK	Alta
RF-04	El sistema debe validar la firma criptográfica de un webhook entrante utilizando el algoritmo correspondiente a cada pasarela y retornar un evento normalizado si la firma es válida.	SDK	Alta
RF-05	El sistema debe lanzar excepciones tipadas con código de error normalizado y mensaje descriptivo ante cualquier condición de fallo esperada (timeout, credenciales inválidas, recurso no encontrado).	SDK	Alta
RF-06	El sistema debe permitir cambiar la pasarela activa modificando únicamente el parámetro de configuración en la inicialización del framework, sin modificar el código de negocio del llamador.	SDK	Alta

ID	Descripción del requisito	Componente	Prioridad
RF-07	El sistema debe reintentar automáticamente una operación ante errores transitorios (timeout, respuestas 5xx) siguiendo una estrategia de backoff exponencial configurable, hasta un número máximo de intentos definido por el usuario.	SDK	Media
RF-08	El sistema debe garantizar que las credenciales de cada pasarela no sean expuestas en logs, mensajes de error ni en las respuestas del SDK.	SDK	Alta
RF-09	El sistema debe exponer endpoints REST que repliquen el comportamiento de creación de pago de cada pasarela soportada, accesibles sin credenciales reales.	API Simulación	Alta
RF-10	El sistema debe permitir configurar el escenario de respuesta de cada endpoint mediante un parámetro de la solicitud, soportando al menos: APROBADO, RECHAZADO, FONDOS_INSUFICIENTES, TIMEOUT y ERROR_RED.	API Simulación	Alta
RF-11	El sistema debe replicar la estructura exacta de las respuestas de error de cada pasarela en los escenarios de fallo configurados.	API Simulación	Alta
RF-12	El sistema debe permitir solicitar el envío de un webhook simulado hacia una URL destino, con el formato y firma correspondientes a la pasarela indicada.	API Simulación	Media
RF-13	El sistema debe exponer la especificación OpenAPI 3.0 de la API de simulación en un endpoint público (/docs), accesible a través de un navegador web.	API Simulación	Media
RF-14	El sistema debe proveer documentación de datos de prueba organizada por pasarela, incluyendo tarjetas de prueba, credenciales de sandbox y referencias de pago válidas para cada proveedor.	Documentación	Alta
RF-15	El sistema debe indicar el comportamiento esperado de cada dato de prueba (tarjeta aprobada, tarjeta rechazada, fondos insuficientes) junto con la fuente oficial del proveedor.	Documentación	Media

Tabla 5. Requisitos funcionales del framework Kit Pagos Colombia.

6.2 Historias de Usuario

Las siguientes historias de usuario complementan los requisitos funcionales de la Tabla 5, expresando cada necesidad desde la perspectiva del usuario y acompañándola de criterios de aceptación verificables. Cada historia referencia el requisito funcional que desarrolla.

6.3 SDK — Interfaz unificada de integración

6.3.1 HU-01: Crear una intención de pago con una pasarela específica

Como	desarrollador de software
Quiero	crear una intención de pago especificando únicamente la pasarela deseada, el monto y los datos del pagador mediante una sola llamada al SDK
Para	no tener que conocer los detalles de autenticación y formato de solicitud propios de cada proveedor

Tabla 6. HU-01

Criterios de aceptación:

- El SDK acepta una solicitud de creación de pago con los parámetros: pasarela, monto, moneda, referencia de orden y datos del pagador.
- El SDK retorna una respuesta normalizada con el identificador de la transacción, el estado inicial y la URL de redirección o token de pago según corresponda.
- Si la pasarela retorna un error, el SDK lanza una excepción tipada con código de error normalizado y mensaje descriptivo.
- El SDK soporta las pasarelas Wompi, PayU, Mercado Pago y Kushki sin que el código del llamador deba cambiar entre ellas.

6.3.2 HU-02: Consultar el estado de una transacción

Como	desarrollador de software
Quiero	consultar el estado actual de una transacción utilizando su identificador, independientemente de la pasarela que la procesó
Para	obtener siempre un estado normalizado sin tener que interpretar los estados específicos de cada proveedor

Tabla 7. HU-02

Criterios de aceptación:

- El SDK retorna el estado de la transacción mapeado a uno de los estados normalizados: PENDIENTE, APROBADO, RECHAZADO, EXPIRADO o ERROR.
- La respuesta incluye el estado original de la pasarela como campo adicional para referencia.
- Si el identificador no existe en la pasarela indicada, el SDK lanza una excepción tipada con código NOT_FOUND.

6.3.3 HU-03: Validar y procesar un webhook entrante

Criterios de aceptación:

- El SDK valida la firma del webhook utilizando el algoritmo correspondiente a cada pasarela (HMAC-SHA256 para Wompi y Kushki, firma propia de PayU y Mercado Pago).
- Si la firma es inválida, el SDK retorna un error tipado con código INVALID_SIGNATURE.

Como	desarrollador de software
Quiero	validar la firma de un webhook recibido y extraer el evento normalizado que contiene, con una sola llamada al SDK
Para	no tener que implementar la lógica de validación de firma de cada pasarela por separado

Tabla 8. HU-03

- Si la firma es válida, el SDK retorna un objeto de evento normalizado con el tipo de evento, el identificador de transacción y el nuevo estado.

6.3.4 HU-04: Cambiar de pasarela sin modificar la lógica de negocio

Como	desarrollador de software senior
Quiero	cambiar la pasarela activa de mi aplicación modificando únicamente la configuración del framework, sin tocar el código de negocio
Para	reducir el riesgo y el esfuerzo de migración entre proveedores

Tabla 9. HU-04

Criterios de aceptación:

- El framework permite especificar la pasarela activa mediante un parámetro de configuración en la inicialización.
- Al cambiar la pasarela en la configuración, todas las operaciones de pago se redirigen al nuevo proveedor sin modificar el código del llamador.
- El cambio de pasarela no afecta la estructura ni el tipo de las respuestas retornadas por el SDK.

6.3.5 HU-05: Manejo automático de reintentos ante fallos transitorios

Como	desarrollador de software
Quiero	que el SDK reintente automáticamente una operación cuando la pasarela devuelve un error transitorio (timeout, error 500)
Para	mejorar la resiliencia de la integración sin implementar lógica de reintento manualmente

Tabla 10. HU-05

Criterios de aceptación:

- El SDK reintenta la operación hasta un máximo configurable de veces ante errores de red o respuestas 5xx de la pasarela.
- Si se supera el número máximo de reintentos, el SDK lanza una excepción tipada con código MAX_RETRIES_EXCEEDED.

6.4 API web de simulación

6.4.1 HU-06: Simular una transacción aprobada

Criterios de aceptación:

Como	desarrollador o tester
Quiero	enviar una solicitud de pago a la API de simulación y recibir una respuesta que replica el comportamiento de una transacción aprobada por cada pasarela
Para	probar el flujo de éxito de mi integración sin depender del sandbox real de la pasarela

Tabla 11. HU-06

- La API de simulación expone un endpoint por pasarela que acepta los mismos parámetros que la API real del proveedor.
- La respuesta del endpoint de éxito replica la estructura exacta de una respuesta aprobada de la pasarela correspondiente.
- La API de simulación es accesible públicamente sin requerir credenciales reales ni configuración local.

6.4.2 HU-07: Simular escenarios de error controlados

Como	tester de software
Quiero	provocar escenarios de rechazo, timeout y error de red en la API de simulación mediante parámetros de la solicitud
Para	verificar que mi integración maneja correctamente los casos de fallo sin tener que reproducirlos en el sandbox real

Tabla 12. HU-07

Criterios de aceptación:

- La API de simulación soporta al menos los siguientes escenarios configurables por parámetro: APROBADO, RECHAZADO, FONDOS_INSUFICIENTES, TIMEOUT, ERROR_RED.
- La respuesta de cada escenario replica la estructura de error real de la pasarela correspondiente.
- El escenario TIMEOUT introduce un retardo configurable antes de responder, simulando latencia real.

6.4.3 HU-08: Recibir una notificación webhook simulada

Como	desarrollador de software
Quiero	solicitar a la API de simulación que envíe un webhook a una URL de mi aplicación, replicando el formato y firma de la pasarela correspondiente
Para	probar el endpoint receptor de webhooks de mi aplicación sin esperar eventos reales del sandbox

Tabla 13. HU-08

Criterios de aceptación:

- La API de simulación acepta una URL destino y un tipo de evento, y envía un webhook firmado replicando el formato de la pasarela indicada.
- La firma del webhook simulado es válida y puede ser verificada con el mecanismo de validación del SDK.
- La API de simulación retorna el resultado del intento de envío (éxito o fallo de la entrega).

6.5 Documentación de datos de prueba

6.5.1 HU-09: Acceder a datos de prueba por pasarela

Como	desarrollador o tester
Quiero	encontrar en un único lugar los datos de prueba disponibles para cada pasarela: tarjetas, credenciales de sandbox, referencias y parámetros especiales
Para	no tener que buscar en múltiples fuentes de documentación de cada proveedor durante el desarrollo

Tabla 14. HU-09

Criterios de aceptación:

- El repositorio del framework incluye una sección de documentación de datos de prueba organizada por pasarela.
- Para cada pasarela se documentan: tarjetas de prueba con sus comportamientos esperados, credenciales de acceso al sandbox, y referencias de pago válidas.
- La documentación indica claramente la fuente oficial de cada dato (enlace a la documentación del proveedor).
- La documentación se mantiene actualizada con los datos de prueba vigentes de cada proveedor al momento de la entrega del proyecto.

7. Requisitos No Funcionales

Los requisitos no funcionales del framework se especifican siguiendo la clasificación FURPS+ (Functionality, Usability, Reliability, Performance, Supportability y restricciones adicionales), adaptada al contexto de una librería de software para desarrolladores.

7.1 Funcionalidad (Functionality)

- El framework debe soportar las operaciones de creación de pago, consulta de estado y validación de webhooks para las cuatro pasarelas integradas (Wompi, PayU, Mercado Pago, Kushki).
- El manejo de credenciales debe garantizar que las claves de API de cada pasarela no sean expuestas en logs, mensajes de error ni en las respuestas del SDK.
- La comunicación entre el SDK y las APIs de las pasarelas debe realizarse exclusivamente a través de HTTPS.
- La validación de webhooks debe verificar la firma criptográfica utilizando el algoritmo definido por cada proveedor antes de procesar el evento.

7.2 Usabilidad (Usability)

- El framework debe poder instalarse mediante un único comando npm sin dependencias de configuración adicionales del sistema operativo. En caso de que el paquete no se encuentre publicado en el registro oficial de npm al momento de la evaluación, la instalación debe ser igualmente posible mediante un único comando apuntando directamente al repositorio público de GitHub del proyecto.

- La documentación del framework debe permitir que un desarrollador con conocimientos básicos de Node.js complete su primera integración siguiendo la guía de inicio rápido en un tiempo máximo de 2 horas, verificado mediante una sesión de prueba de usabilidad con un participante externo al equipo de desarrollo durante la fase de evaluación del proyecto.
- Los mensajes de error del SDK deben ser descriptivos, en español e inglés, e incluir el código de error, la pasarela involucrada y una sugerencia de resolución cuando sea posible.
- La API de simulación debe incluir documentación OpenAPI 3.0 accesible públicamente para facilitar su exploración con herramientas como Postman o Swagger UI.

7.3 Confiabilidad (Reliability)

- El mecanismo de reintentos del SDK debe implementar backoff exponencial con un máximo configurable de intentos (valor por defecto: 3 reintentos con intervalos de 1s, 2s y 4s).
- El SDK no debe lanzar excepciones no manejadas bajo ninguna condición de error esperada (respuesta inválida de la pasarela, timeout de red, credenciales incorrectas).
- La API de simulación debe permanecer desplegada y accesible de forma continua durante todo el período de evaluación del proyecto, sin interrupciones planificadas. No se establece un porcentaje formal de disponibilidad dado que el proyecto no cuenta con infraestructura de monitoreo activo ni acuerdos de nivel de servicio, sin embargo cualquier interrupción no planificada debe ser resuelta en un plazo máximo de 24 horas por el equipo de desarrollo.

7.4 Rendimiento (Performance)

- El overhead introducido por el framework frente a una llamada directa a la API de cada pasarela no debe superar los 100 milisegundos en condiciones de red normales, medido mediante pruebas automatizadas ejecutadas en un entorno local controlado utilizando Jest con soporte de mocks de red.
- La API de simulación debe responder en menos de 500 milisegundos para los escenarios que no simulan latencia intencionada, medido desde el envío de la solicitud hasta la recepción de la respuesta completa utilizando Postman o k6 desde una conexión de red estándar. Este umbral se establece considerando que el despliegue se realizará sobre infraestructura de nivel gratuito, donde los tiempos de respuesta están sujetos a las condiciones del proveedor de hospedaje.
- El SDK no debe mantener conexiones persistentes hacia las pasarelas entre operaciones para evitar el consumo innecesario de recursos del proceso huésped.

7.5 Soporte y mantenibilidad (Supportability)

- El framework debe seguir la arquitectura hexagonal (Ports & Adapters), de modo que la incorporación de una nueva pasarela requiera únicamente implementar una nueva clase adaptador sin modificar el núcleo.
- El repositorio público del framework debe incluir instrucciones de contribución (`CONTRIBUTING.md`) que describan cómo implementar un nuevo adaptador.
- Cada adaptador debe estar acompañado de pruebas de integración que verifiquen su funcionamiento contra la API de simulación.

7.6 Restricciones adicionales (+)

- Restricción de diseño: el framework se implementa en TypeScript con target ES2020. El código compilado debe ser compatible con Node.js versión 18 LTS en adelante, por ser la versión con soporte activo de largo plazo más ampliamente adoptada al momento del desarrollo del proyecto.
- Restricción de distribución: el framework se distribuye como paquete npm bajo la licencia Apache 2.0. El código fuente se publica en un repositorio público de GitHub.
- Restricción de seguridad: el framework no debe almacenar datos de tarjeta de crédito en memoria más tiempo del estrictamente necesario para realizar la operación, y no debe registrar (log) ningún dato sensible del pagador.
- Restricción regulatoria: el framework actúa exclusivamente como intermediario que delega las operaciones hacia cada pasarela sin modificar, interceptar ni almacenar los datos sensibles del proceso de pago. La responsabilidad sobre el cumplimiento de PCI DSS recae en las pasarelas integradas y en las aplicaciones que consumen el framework, no en el framework mismo.

8. Referencias

- IEEE. (1998). *IEEE recommended practice for software requirements specifications* (IEEE Std 830-1998). `https://doi.org/10.1109/IEEESTD.1998.88286`
- Peffers, K., Tuunanen, T., Rothenberger, M., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45-77.
- Cockburn, A. (2024). *Hexagonal architecture explained*. `https://alistair.cockburn.us/hexagonal-architecture/`
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Chidamber, S. R., & Kemerer, C. F. (1994). A metrics suite for object-oriented design. *IEEE Transactions on Software Engineering*, 20(6), 476-493. `https://doi.org/10.1109/32.295895`
- Hohpe, G., & Woolf, B. (2003). *Enterprise integration patterns: Designing, building, and deploying messaging solutions*. Addison-Wesley.
- Hisyam, M., & Manuaba, I. B. K. (2022). Integration model of multiple payment gateways for online split payment scenario. *2022 International Conference on Information Management and Technology (ICIMTech)*, 122-126. `https://doi.org/10.1109/ICIMTech55957.2022.9915168`
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Tesis doctoral, University of California, Irvine).
- Ammann, P., & Offutt, J. (2016). *Introduction to software testing*. Cambridge University Press. `https://doi.org/10.1017/9781316771273`